

DATA MINING

GUIBERT Thomas

&

HAMILTON Djibril

16/05/2025

OBJECTIVES

1.Data Preparation.

VSCODE request:

```
# Required Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Load the data
df = pd.read_csv('energy_consumption_data.csv')

# Display basic information
print(df.info())
print(df.describe())

# Check for missing values
print(df.isnull().sum())
```

✓ 4.5s

Result :

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   household_size                        10000 non-null  int64
1   house_area                           10000 non-null  float64
2   num_appliances                       10000 non-null  int64
3   avg_temp                             10000 non-null  float64
4   region                               10000 non-null  object
5   region_encoded                       10000 non-null  int64
6   ac_hours_per_day                     10000 non-null  float64
7   heater_hours_per_day                 10000 non-null  float64
8   has_solar                           10000 non-null  int64
9   income_level                         10000 non-null  object
10  income_encoded                       10000 non-null  int64
11  working_hours_away                   10000 non-null  float64
12  energy_efficiency_score               10000 non-null  int64
13  avg_daily_energy_consumption          10000 non-null  float64
dtypes: float64(6), int64(6), object(2)
memory usage: 1.1+ MB
None
   household_size  house_area  num_appliances  avg_temp  \
count  10000.000000  10000.000000  10000.000000  10000.000000
mean         3.499900    120.386491     10.056500    20.069088
...
working_hours_away      0
energy_efficiency_score  0
avg_daily_energy_consumption  0
dtype: int64
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

Explanation:

The dataset is imported and examined to understand its overall structure. We check the types of variables, get statistical summaries like mean and standard deviation, and verify whether any columns contain missing values. This step ensures the data is clean and usable for analysis

VSCODE request:

```

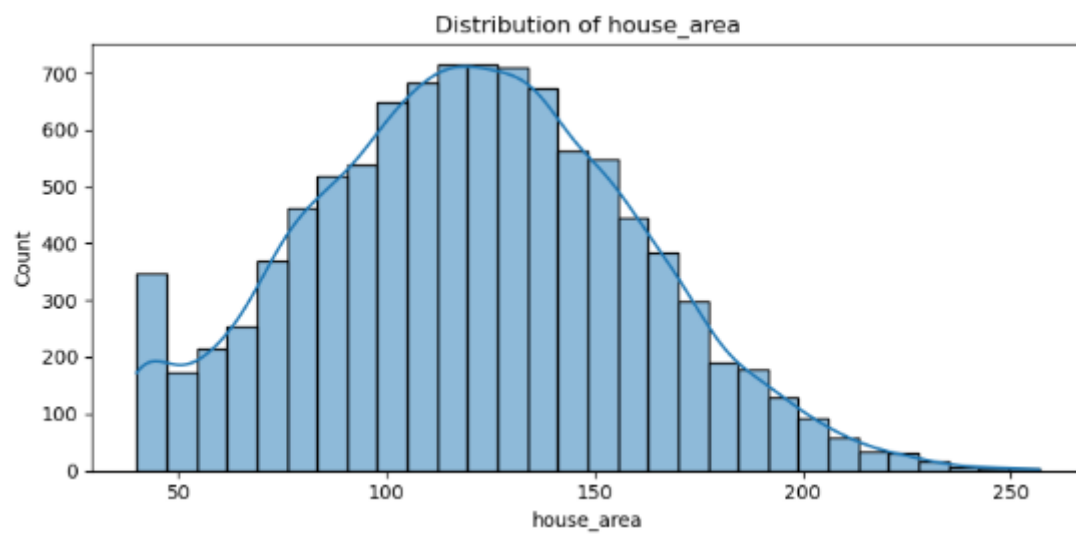
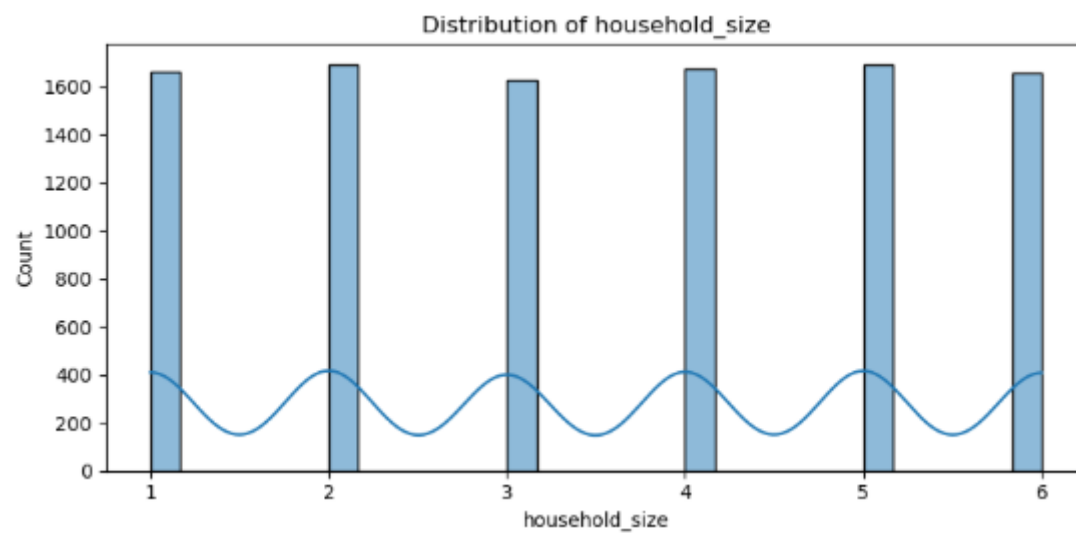
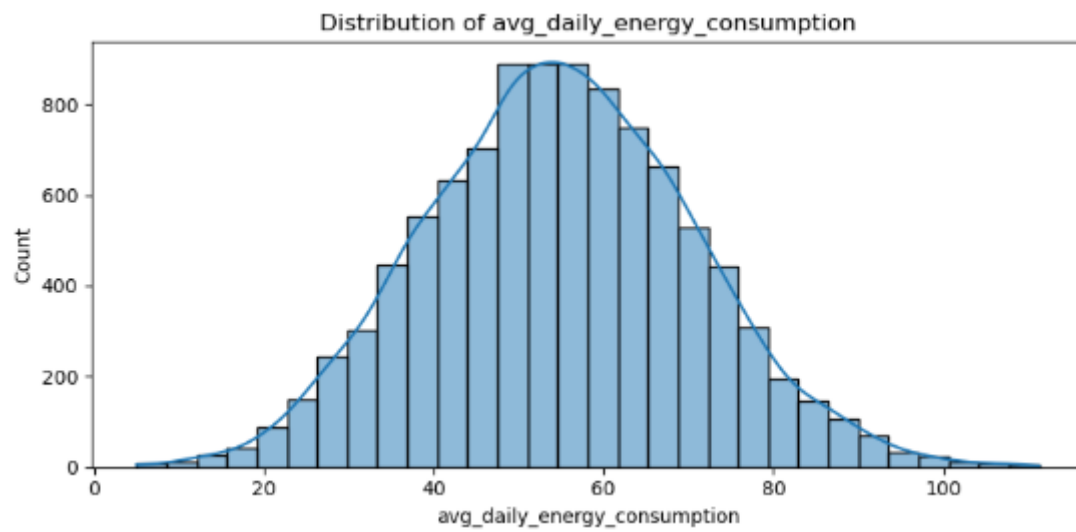
num_features = [
    'avg_daily_energy_consumption', 'household_size', 'house_area',
    'num_appliances', 'ac_hours_per_day', 'heater_hours_per_day',
    'energy_efficiency_score'
]

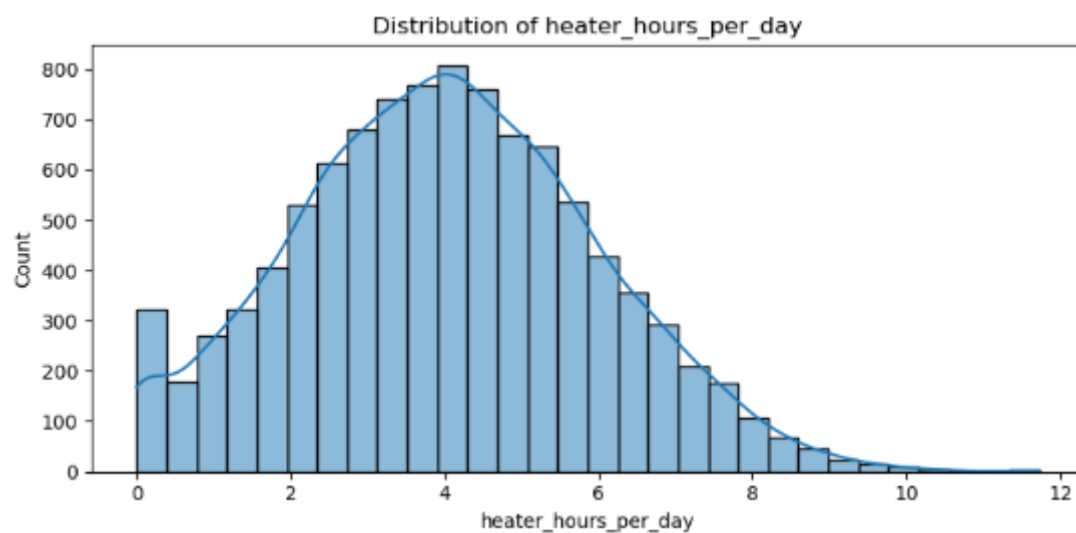
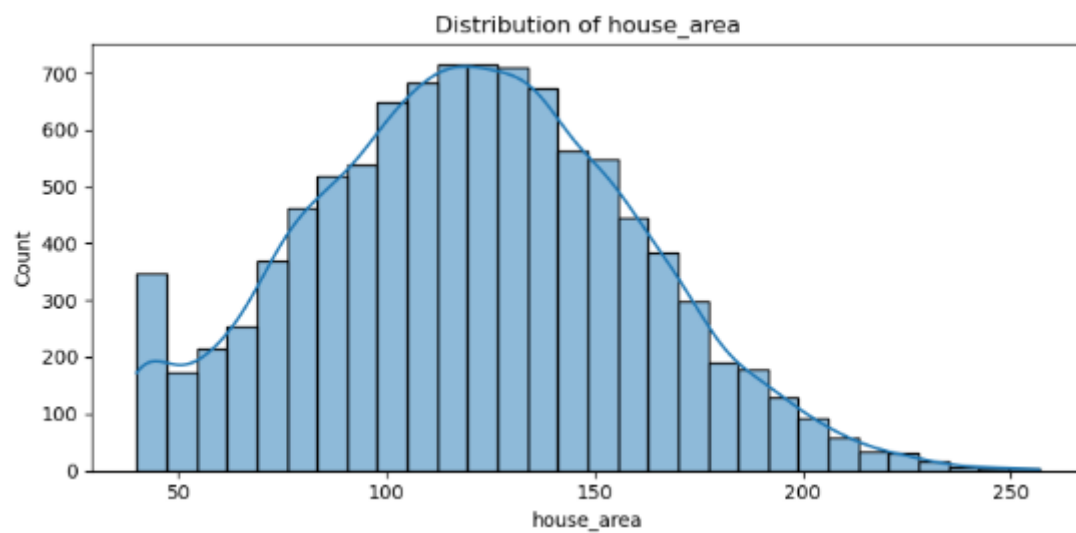
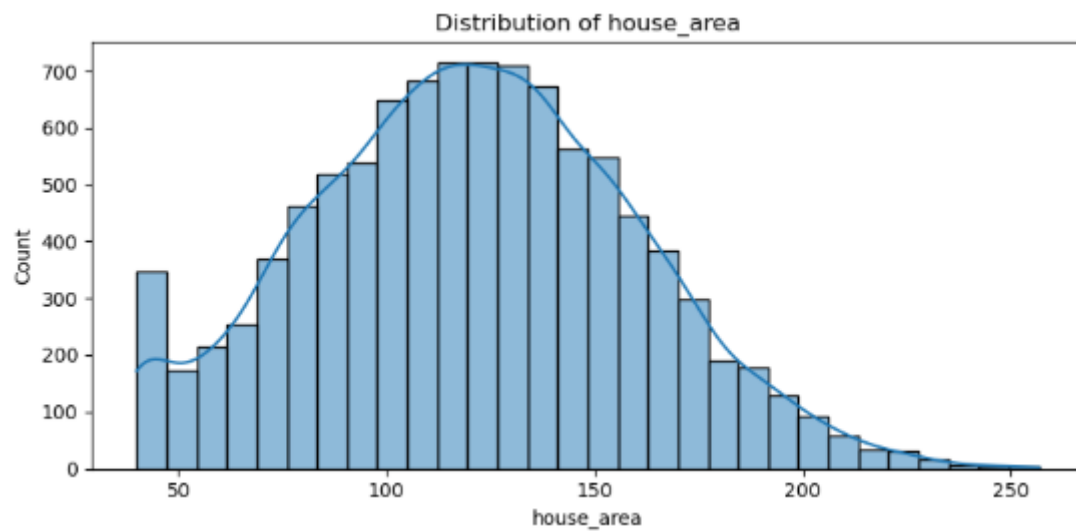
# Histograms
for feature in num_features:
    plt.figure(figsize=(8, 4))
    sns.histplot(df[feature], kde=True, bins=30)
    plt.title(f'Distribution of {feature}')
    plt.xlabel(feature)
    plt.tight_layout()
    plt.show()

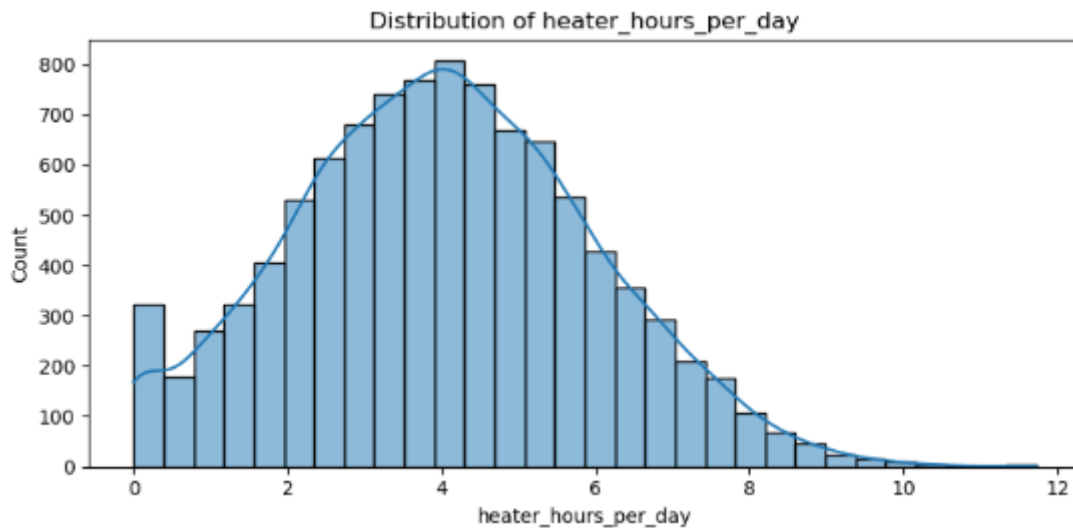
```

✓ 3.2s

Result:







Explanation:

This section of the code is used to visualize the distribution of several numerical features in the dataset. First, a list called `num_features` is created. It contains the names of columns that represent numerical characteristics of each household, such as average daily energy consumption, household size, house area, number of appliances, daily air conditioner and heater usage, and energy efficiency score. Then, the code uses a loop to go through each feature in the list. For each one, it creates a histogram using the `sns.histplot()` function. This plot shows how frequently different values appear in the dataset. The `kde=True` option adds a smooth curve over the bars to show the overall shape of the distribution. Each plot includes a title and a label for the x-axis, which corresponds to the feature being displayed. The plots are arranged neatly using `tight_layout()` and shown one by one with `plt.show()`. This visual analysis helps to understand the shape of the data for each feature, such as whether values are spread out, concentrated, or skewed in a particular direction.

2. Use visualization techniques to analyze the features that most affect energy consumption (e.g., home size, appliance count, heating/cooling hours).

VSCODE request:

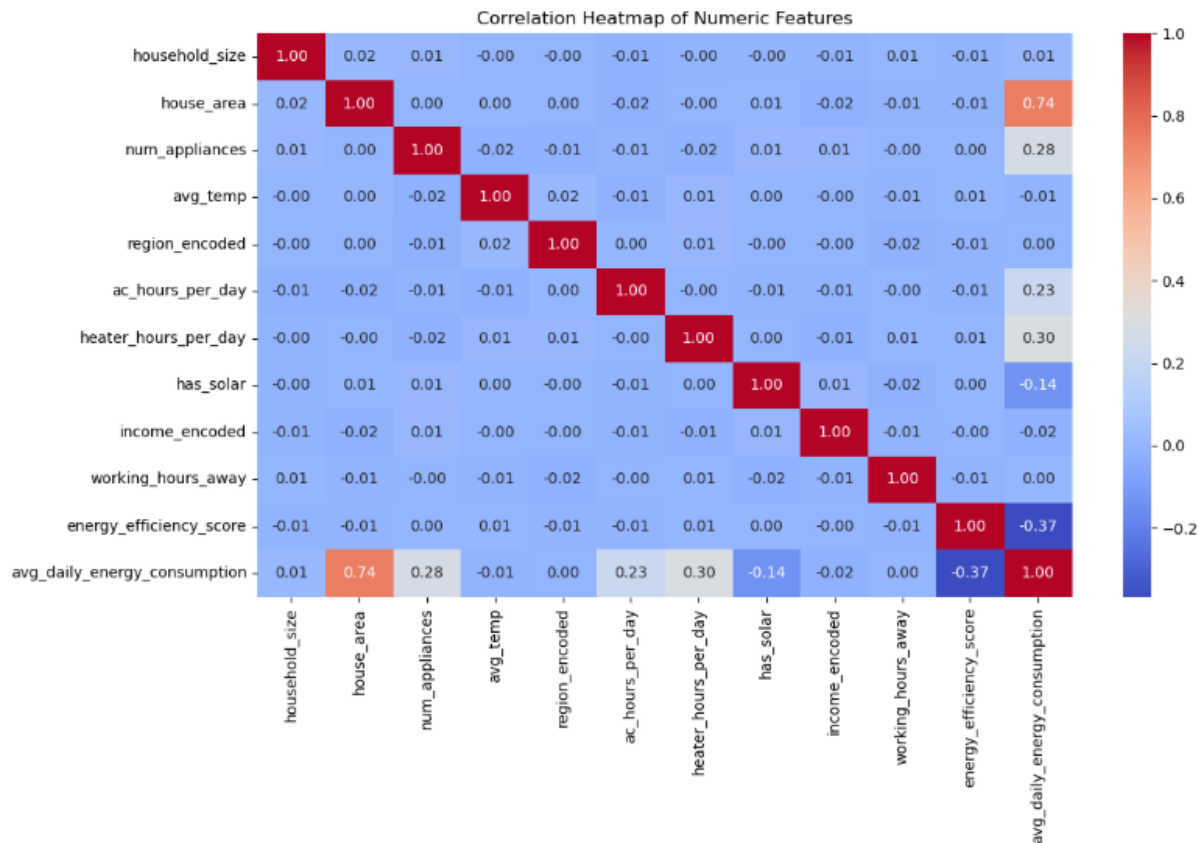
```
# Step 1: Visualization - Correlation Heatmap

numeric_df = df.select_dtypes(include=['number'])

plt.figure(figsize=(12, 8))
sns.heatmap(numeric_df.corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Heatmap of Numeric Features")
plt.tight_layout()
plt.show()

✓ 3.4s
```

Result:



Explanation:

This code creates a heatmap to show how numerical features in the dataset are related. It first selects only the numeric columns and stores them in `numeric_df`. Then, it calculates the correlation between each pair of features. Using `sns.heatmap()`, it plots these correlations, showing the values on the grid with colors indicating the strength and direction of each relationship. Red means strong positive correlation, blue means negative. Finally, the plot is titled and displayed. This helps identify which features are strongly related.

3. Use clustering methods to identify types of households with similar consumption patterns (e.g., “energy-efficient small homes”, “large inefficient homes”, etc.).

VSCODE request:

```

df = pd.read_csv("energy_consumption_data.csv")

sample_df = df.sample(n=1000, random_state=42)

# Features used for clustering
features = [
    'household_size', 'house_area', 'avg_temp', 'ac_hours_per_day', 'heater_hours_per_day',
    'has_solar', 'income_encoded', 'working_hours_away', 'energy_efficiency_score'
]
X_sample = sample_df[features]

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_sample)

# Reduce dimensions
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Cluster using KMeans
kmeans = KMeans(n_clusters=4, random_state=42)
clusters = kmeans.fit_predict(X_scaled)

# Add cluster labels to original sample
sample_df['cluster'] = clusters

# Group by cluster and show mean feature values
cluster_summary = sample_df.groupby('cluster')[features].mean().round(2)
print(cluster_summary)

# Plot
plt.figure(figsize=(10, 6))
sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=clusters,
                palette='Set2', s=60, edgecolor='black')
plt.title("Clustered Households")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.grid(True)
plt.legend(title="Cluster")
plt.tight_layout()
plt.show()

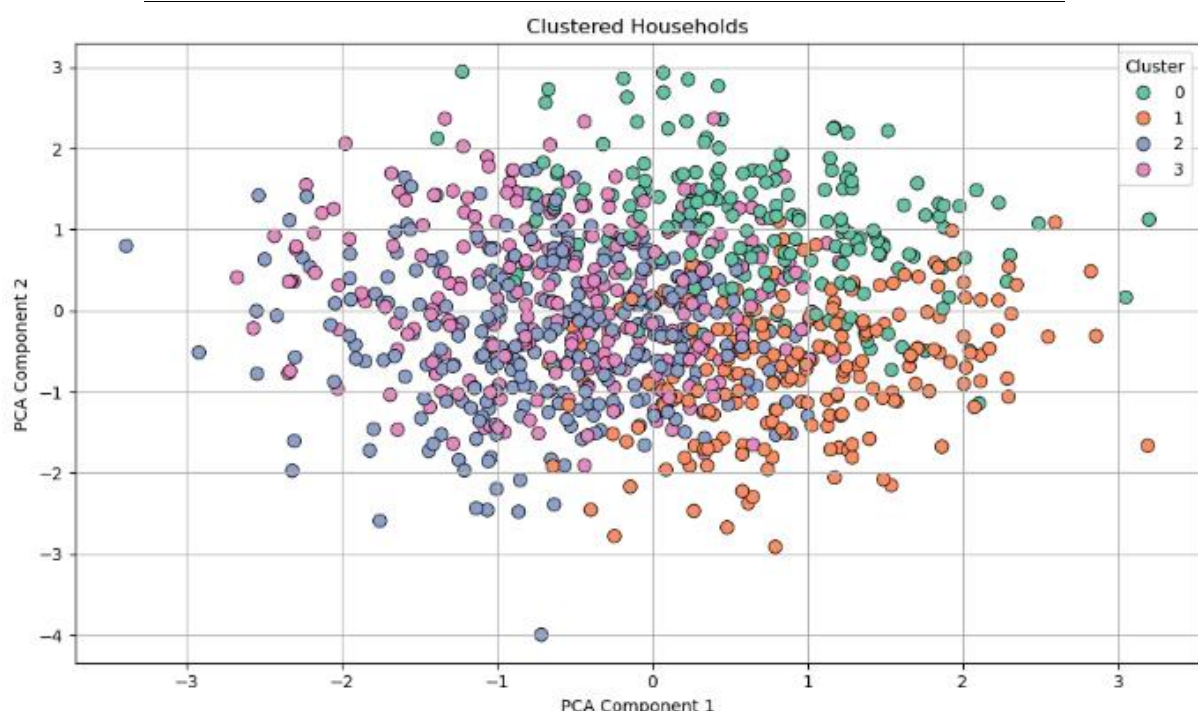
```

Result:

	household_size	house_area	avg_temp	ac_hours_per_day	\
cluster					
0	2.86	116.26	21.53	2.96	
1	3.60	127.55	21.85	3.17	
2	4.97	124.84	18.25	3.11	
3	2.00	113.07	19.50	3.06	

	heater_hours_per_day	has_solar	income_encoded	working_hours_away	\
cluster					
0	6.09	0.12	1.25	5.94	
1	4.85	0.38	0.34	4.68	
2	2.99	0.30	0.85	6.45	
3	3.07	0.35	0.94	7.21	

	energy_efficiency_score
cluster	
0	3.86
1	7.44
2	4.62
3	6.63



Explanation:

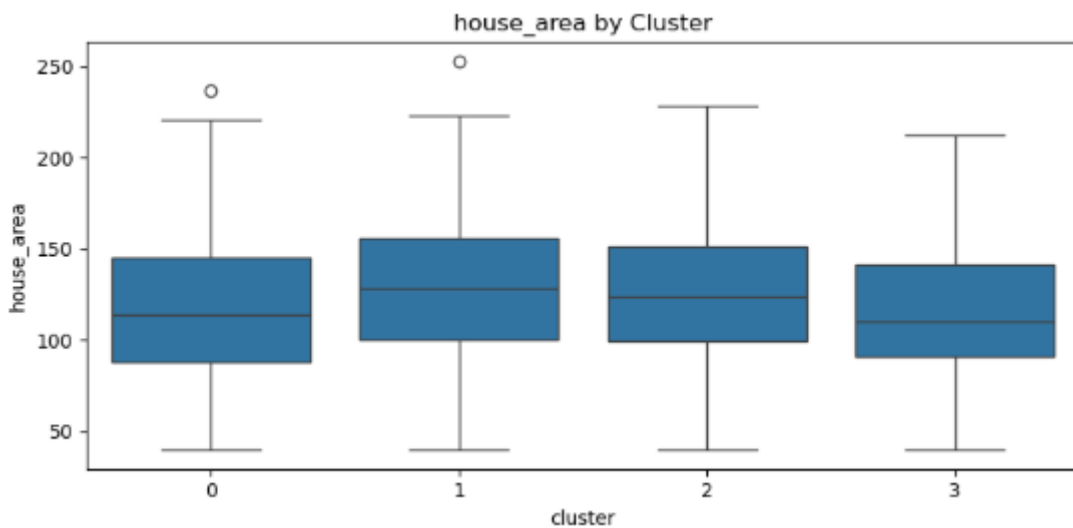
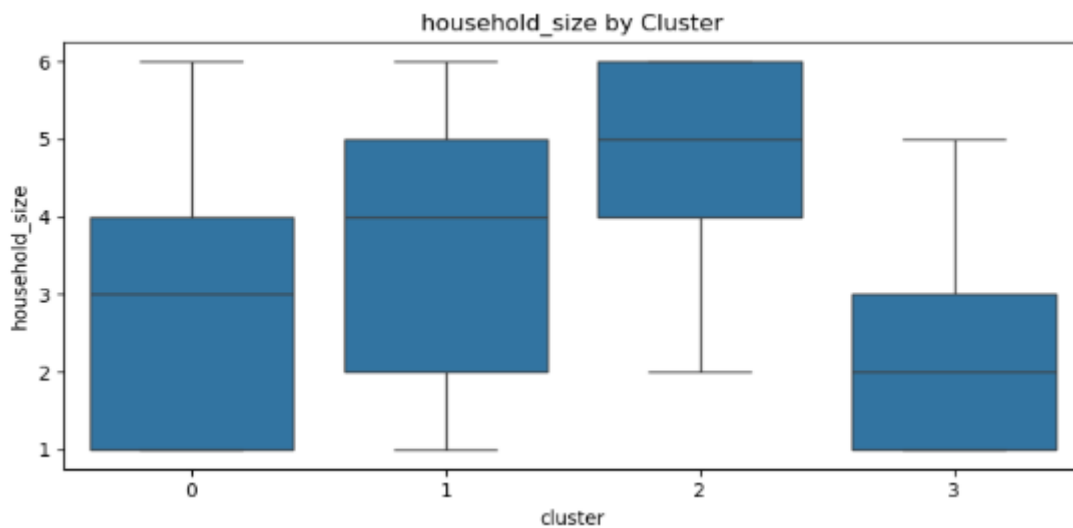
It selects key features like household size, house area, temperature, AC/heater usage, solar panel presence, income, working hours away, and energy efficiency score. These features describe each household's energy behavior. To prepare the data for clustering, it standardizes all features, so they have the same scale, which is crucial for algorithms like KMeans that rely on distances. Then, it reduces the data to 2 dimensions using PCA (Principal Component Analysis) to make it easier to visualize and interpret. The KMeans algorithm is then applied to divide households into 4 clusters based on their standardized characteristics. Each household is labeled with its assigned cluster, and the average values of each feature are calculated per cluster to understand the differences between them. Finally, the script visualizes the households on a 2D scatter plot using the PCA results. Each point represents a household, and colors indicate the cluster it belongs to. This helps clearly see how households are grouped based on energy-related behaviors.

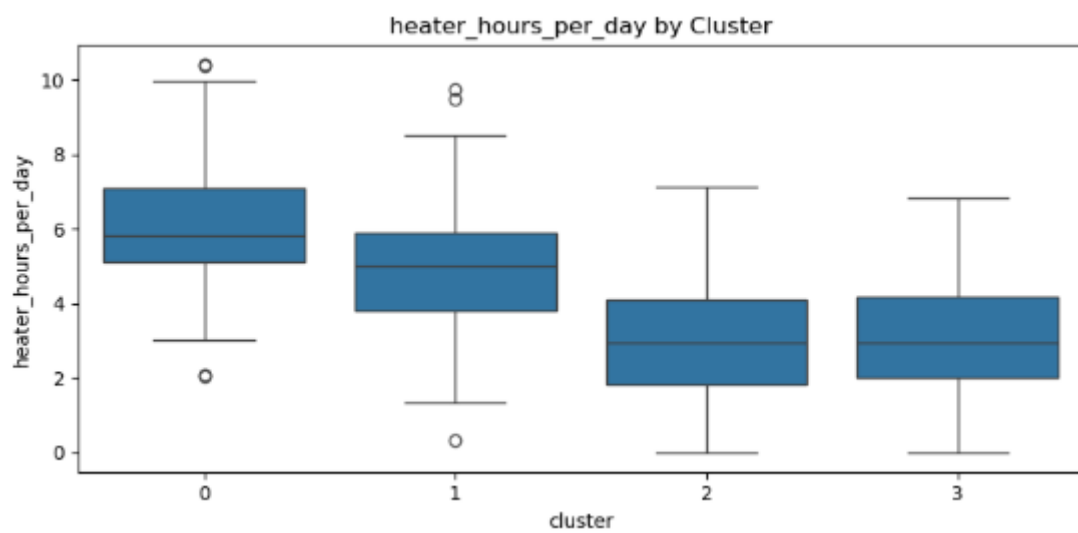
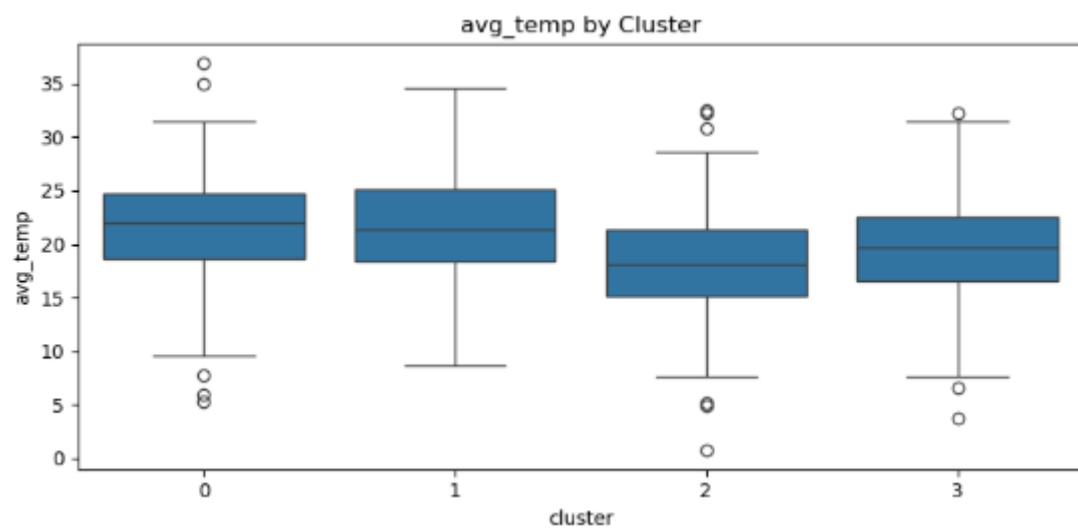
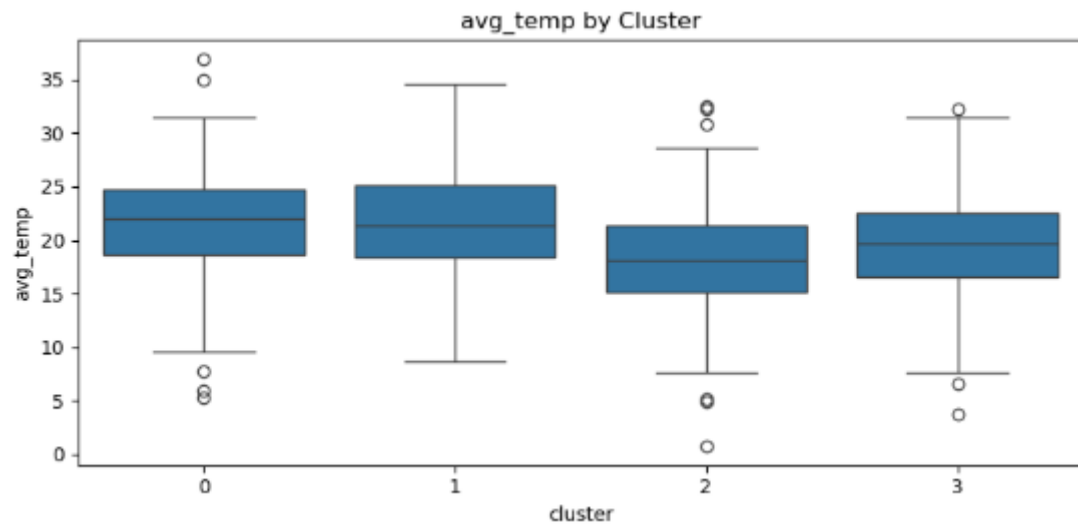
VSCODE request:

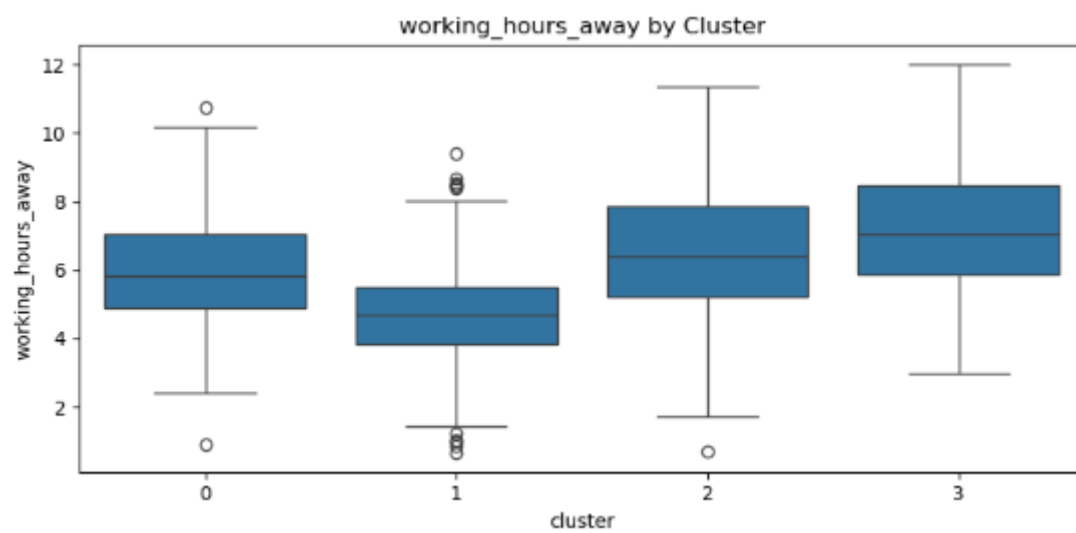
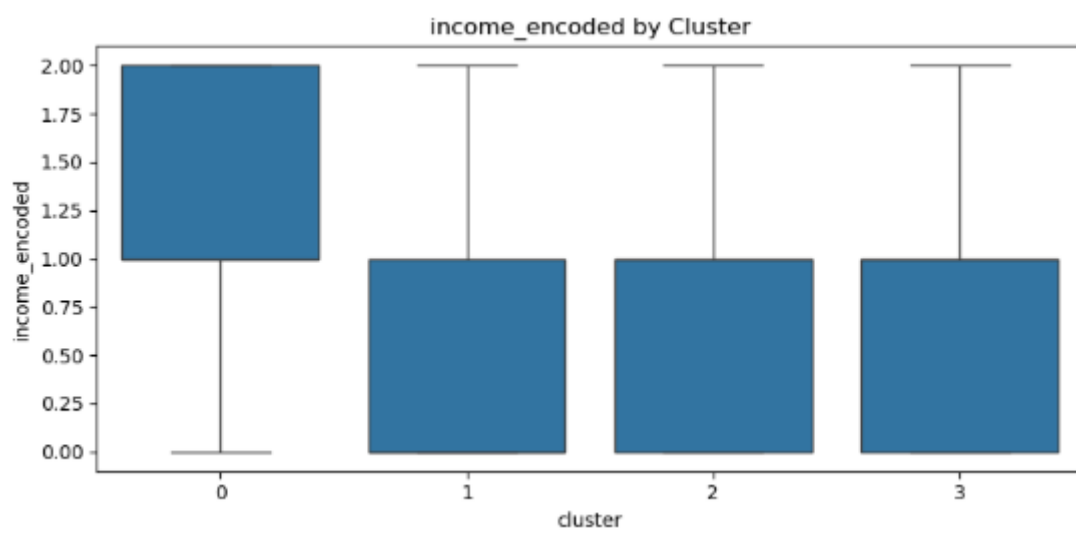
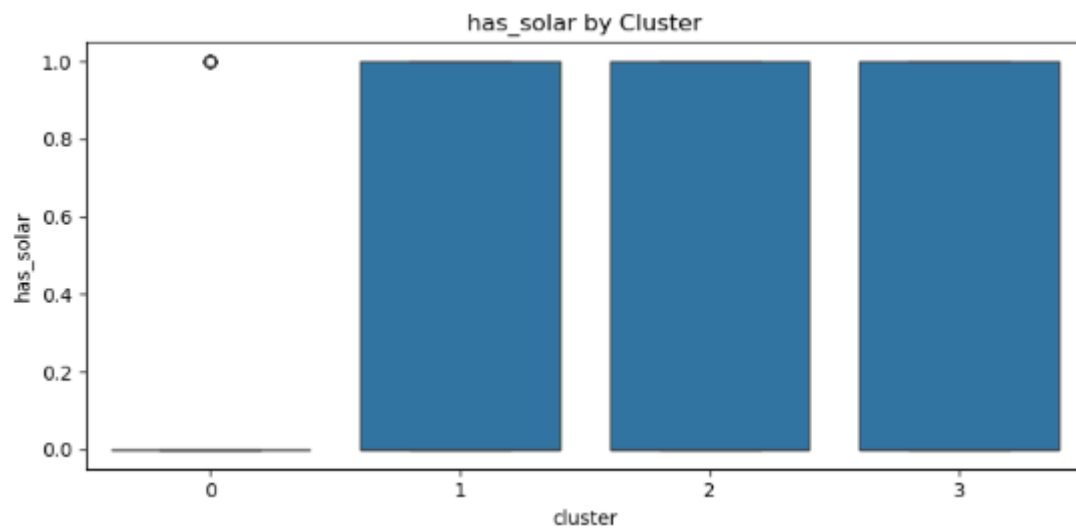
```
for feature in features:
    plt.figure(figsize=(8, 4))
    sns.boxplot(x='cluster', y=feature, data=sample_df)
    plt.title(f'{feature} by Cluster')
    plt.tight_layout()
    plt.show()
```

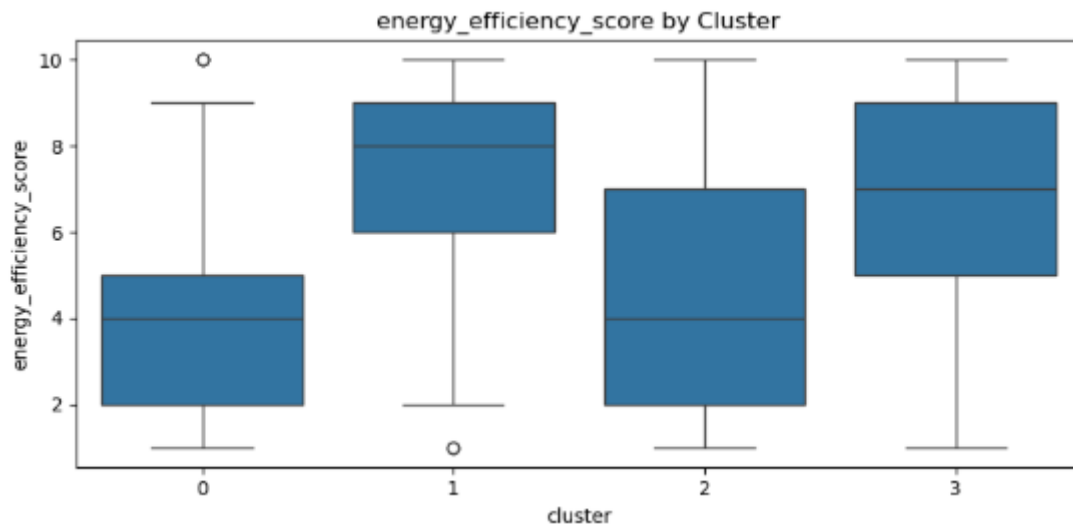
✓ 2.9s

Result:









Explanation:

This code creates a boxplot for each selected feature, grouped by cluster. It loops through all the features and, for each one, it generates a boxplot using `sns.boxplot()`. The x-axis shows the cluster number, and the y-axis shows the values of the feature. Each plot helps compare how the values of that feature vary across different clusters. The box shows the median and spread of the data, while any outliers appear as individual points. Finally, the plot is titled and displayed. This helps to understand how each feature differs between the clusters.

4. Use supervised learning with feedforward neural networks to predict the average daily energy consumption of a household

VSCODE request:

```

x = df[features_for_clustering]
y = df['avg_daily_energy_consumption']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.10, random_state=42
)

scaler_nn = StandardScaler()
x_train_scaled = scaler_nn.fit_transform(x_train)
x_test_scaled = scaler_nn.transform(x_test)

model = Sequential([
    Dense(64, activation='relu', input_shape=(x_train_scaled.shape[1],)),
    Dense(32, activation='relu'),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse', metrics=['mae'])

history = model.fit(x_train_scaled, y_train, epochs=50, batch_size=32, verbose=1)

y_pred = model.predict(x_test_scaled).flatten()

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Neural Network MSE: {mse:.2f}")
print(f"Neural Network R2 Score: {r2:.2f}")

```

✓ 29.3s

Result:

```

super().__init__(activity_regularizer=activity_regularizer, **kwargs)
282/282 ————— 1s 1ms/step - loss: 2353.4255 - mae: 43.8045
Epoch 2/50
282/282 ————— 0s 1ms/step - loss: 57.9835 - mae: 6.0741
Epoch 3/50
282/282 ————— 0s 1ms/step - loss: 48.0219 - mae: 5.5250
Epoch 4/50
282/282 ————— 0s 1ms/step - loss: 39.2629 - mae: 4.9826
Epoch 5/50
282/282 ————— 0s 1ms/step - loss: 32.5347 - mae: 4.5522
Epoch 6/50
282/282 ————— 0s 1ms/step - loss: 27.8390 - mae: 4.1863
Epoch 7/50
282/282 ————— 0s 1ms/step - loss: 25.6770 - mae: 4.0205
Epoch 8/50
282/282 ————— 0s 1ms/step - loss: 22.9377 - mae: 3.7989
Epoch 9/50
282/282 ————— 0s 1ms/step - loss: 21.4954 - mae: 3.6885
Epoch 10/50
282/282 ————— 0s 1ms/step - loss: 18.7801 - mae: 3.4561
Epoch 11/50
282/282 ————— 0s 2ms/step - loss: 17.9031 - mae: 3.3834
Epoch 12/50
282/282 ————— 0s 1ms/step - loss: 17.2196 - mae: 3.3028
Epoch 13/50
282/282 ————— 0s 1ms/step - loss: 16.7742 - mae: 3.2724
...
282/282 ————— 0s 1ms/step - loss: 14.8857 - mae: 3.0773
32/32 ————— 0s 2ms/step
Neural Network MSE: 17.13
Neural Network R2 Score: 0.94

```

Explanation:

This code builds and trains a neural network to predict daily energy consumption. First, it selects the input features (X) and the target variable (y), which is the average daily energy consumption. The data is then split into training and test sets, with 10% of the data used for testing. Next, the features are standardized using StandardScaler to ensure they are on the same scale, which improves model performance. The scaler is fitted on the training data and applied to both training and test sets. A neural network model is then created using Keras. It has two hidden layers: one with 64 neurons and one with 32, both using ReLU activation. The output layer has a single neuron, since it predicts one continuous value. The model is compiled with the Adam optimizer and uses mean squared error (MSE) as the loss function. It also tracks mean absolute error (MAE) during training. The model is trained for 50 epochs with a batch size of 32, and 10% of the training data is used for validation. After training, the model predicts values on the test set. The predictions are evaluated using two metrics: MSE and R² (coefficient of determination), which are printed at the end. The model achieved a low MSE of 17.13, meaning the average squared difference between the predicted and actual energy consumption values is relatively small. Additionally, the high R² score of 0.94 indicates that the model explains 94% of the variance in the target variable. This suggests the neural network has learned the underlying patterns in the data very well and is making highly accurate predictions.

VSCODE request:

```

features = ['household_size', 'house_area', 'num_appliances', 'avg_temp',
            'region_encoded', 'ac_hours_per_day', 'heater_hours_per_day',
            'has_solar', 'income_encoded', 'working_hours_away',
            'energy_efficiency_score']
target = 'avg_daily_energy_consumption'

X = df[features]
y = df[target]

# Split data into train and test sets (90% train, 10% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=42)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Build and train neural network
nn_model = MLPRegressor(hidden_layer_sizes=(50, 30, 10), activation='relu',
                        solver='adam', max_iter=1000, random_state=42)
nn_model.fit(X_train_scaled, y_train)

# Make predictions
y_pred = nn_model.predict(X_test_scaled)

# Evaluate model
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nNeural Network Performance:")
print(f"RMSE: {rmse:.2f}")
print(f"R-squared: {r2:.2f}")

```

```

# Visualize predictions vs actual
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'k--', lw=2)
plt.xlabel('Actual Energy Consumption')
plt.ylabel('Predicted Energy Consumption')
plt.title('Actual vs Predicted Energy Consumption')
plt.show()

# Feature importance analysis (using permutation importance)
from sklearn.inspection import permutation_importance

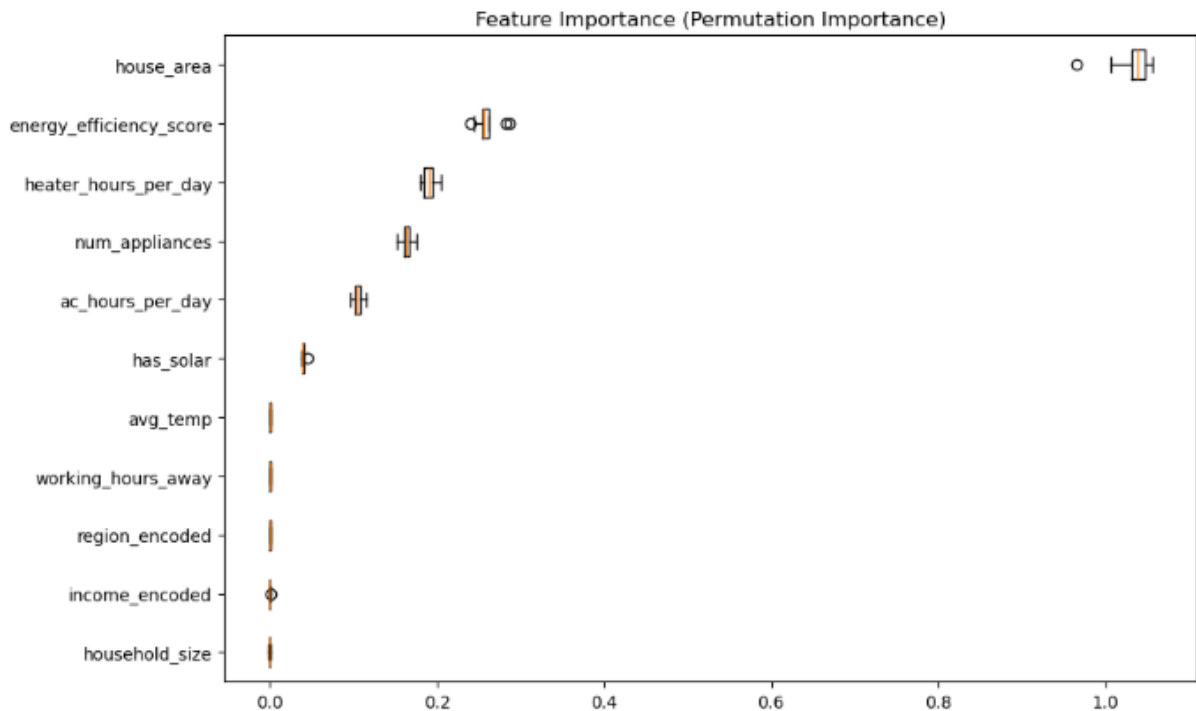
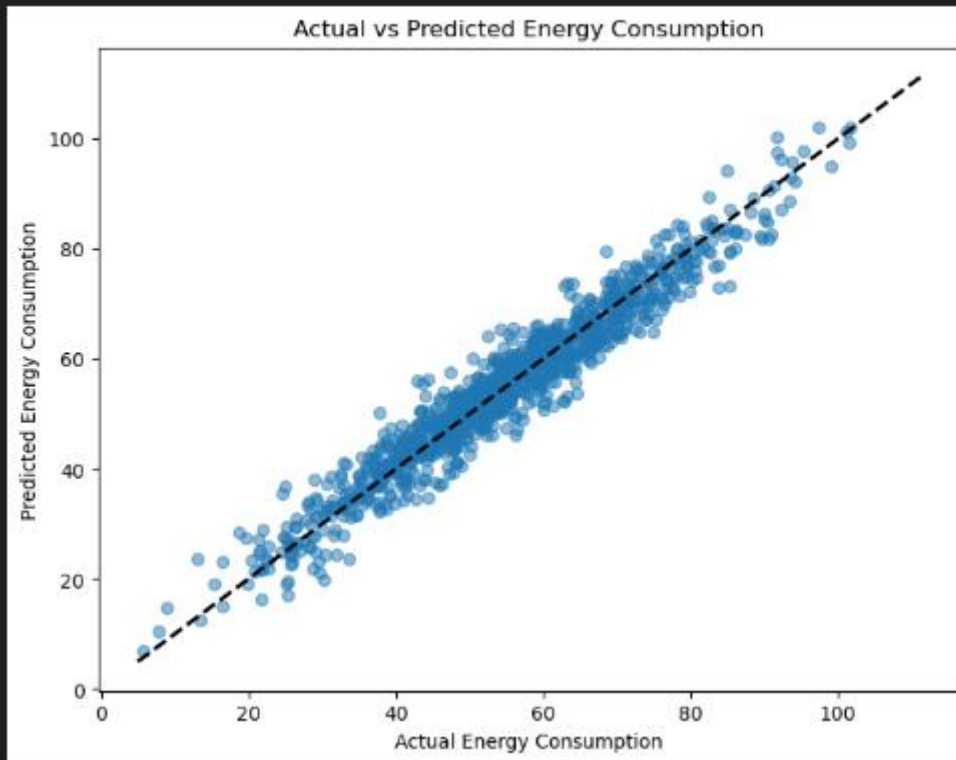
result = permutation_importance(nn_model, X_test_scaled, y_test, n_repeats=10, random_state=42)
sorted_idx = result.importances_mean.argsort()

plt.figure(figsize=(10, 6))
plt.boxplot(result.importances[sorted_idx].T, vert=False, labels=X.columns[sorted_idx])
plt.title("Feature Importance (Permutation Importance)")
plt.tight_layout()
plt.show()

```

Result:

Neural Network Performance:
RMSE: 4.08
R-squared: 0.94



Explanation:

This code trains a neural network to predict a household's average daily energy consumption using several features. First, it defines the input features, such as household size, house area, number of appliances, average temperature, solar panel ownership, and more. The target

variable is `avg_daily_energy_consumption`. The data is split into training and test sets, with 90% used for training and 10% for testing. All features are then standardized using `StandardScaler` to improve model performance. The model used is an `MLPRegressor`, a type of neural network provided by `Scikit-learn`. It has three hidden layers with 50, 30, and 10 neurons, and uses `ReLU` activation. The model is trained using the `Adam` optimizer for up to 1,000 iterations. After training, it makes predictions on the test set. The model's performance is evaluated using `RMSE` (Root Mean Squared Error) and R^2 score. The model achieved an `RMSE` of 4.08, indicating that, on average, its predictions deviate from the actual energy consumption by about 4.08 units. The R^2 score of 0.94 means that 94% of the variability in household energy consumption is explained by the model's input features. This combination of low error and high explanatory power suggests that the neural network is making highly accurate and reliable predictions. A scatter plot is then shown comparing actual vs. predicted energy consumption. A diagonal line is added for reference; points close to this line indicate better predictions. Finally, the code performs a feature importance analysis using permutation importance. This shows which features had the biggest impact on the model's predictions. The results are displayed in a horizontal boxplot for easy interpretation.

VSCODE request:

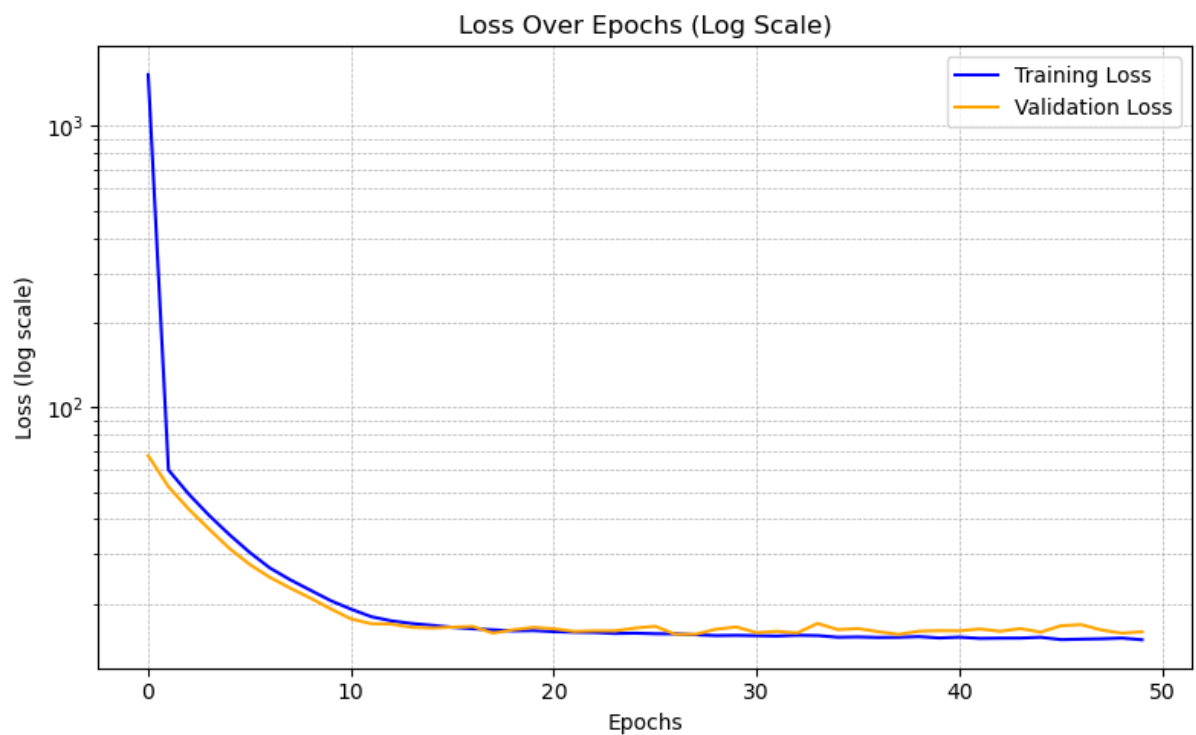
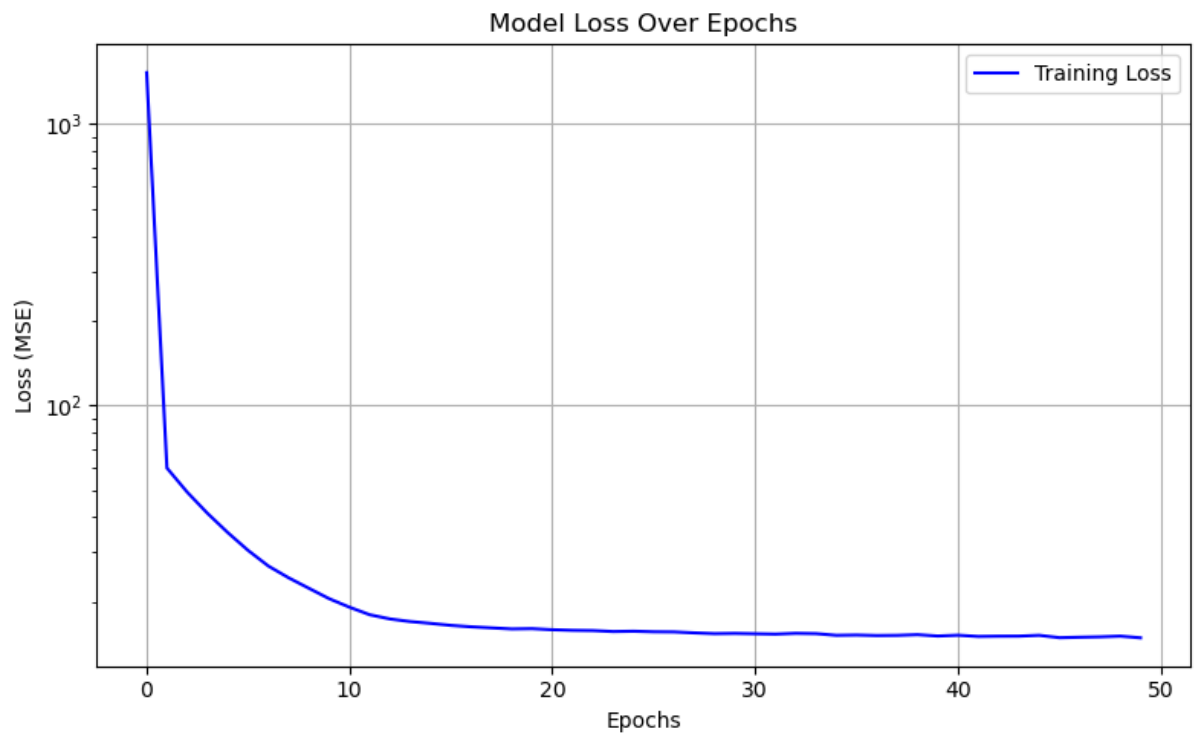
```
# Plot training loss over epochs
plt.figure(figsize=(8, 5))
plt.plot(history.history['loss'], label='Training Loss', color='blue')
plt.yscale('log')
plt.title('Model Loss Over Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss (MSE)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# With validation loss

plt.figure(figsize=(8, 5))
plt.plot(history.history['loss'], label='Training Loss', color='blue')
if 'val_loss' in history.history:
    plt.plot(history.history['val_loss'], label='Validation Loss', color='orange')

plt.yscale('log') # Set y-axis to logarithmic scale
plt.title('Loss Over Epochs (Log Scale)')
plt.xlabel('Epochs')
plt.ylabel('Loss (log scale)')
plt.legend()
plt.grid(True, which="both", linestyle='--', linewidth=0.5)
plt.tight_layout()
plt.show()
```

Result:



Explanation:

This code visualizes how the neural network's loss changed during training. In the first plot, it shows the training loss (mean squared error) for each epoch using a blue line. The y-axis is set to a logarithmic scale to better show changes in loss, especially when values are small. This helps track the model's learning progress over time. In the second plot, it adds the validation loss if available. The training loss is still shown in blue, while the validation loss is shown in orange. Using both curves helps compare how well the model is learning on the training data

versus unseen validation data. The logarithmic scale, gridlines, and labels make it easier to see how the model's performance improves and whether overfitting occurs. The graph shows that both training and validation loss decrease sharply after just 2 epochs and then follow similar downward trends. This indicates that the model quickly learned the underlying patterns in the data and continued to improve steadily. The close alignment of the training and validation curves suggests good generalization with no signs of overfitting.

CONCLUSION:

In this project, we successfully applied a full data mining process to analyze and model household energy consumption. We began with careful data preparation, including handling missing values, visualizing distributions, and understanding feature relationships through a correlation heatmap. Clustering techniques such as PCA and KMeans allowed us to group households based on similar energy usage patterns, revealing insights into different types of consumption behavior (e.g., energy-efficient vs. high-usage homes).

Next, we implemented predictive modeling using neural networks. Two models were tested: one built with Keras and another using Scikit-learn's MLPRegressor. Both were trained on standardized features and evaluated with RMSE and R^2 metrics, confirming their ability to predict daily energy consumption with reasonable accuracy. The Keras model achieved an MSE of 17.13 and an R^2 score of 0.94, while the Scikit-learn model produced a lower RMSE of 4.08 and the same high R^2 score of 0.94. These results indicate strong model performance and accurate predictions. Additionally, loss curves showed smooth and steady learning with no significant overfitting. Feature importance analysis further helped identify the most influential factors affecting energy use, such as household size, appliance count, and solar panel ownership.

Overall, this project demonstrates how data mining can be used to better understand and predict energy behavior at the household level, which could support smarter energy management and policy decisions.